

EXP1: GitHub

25 July 2025 11:49 AM

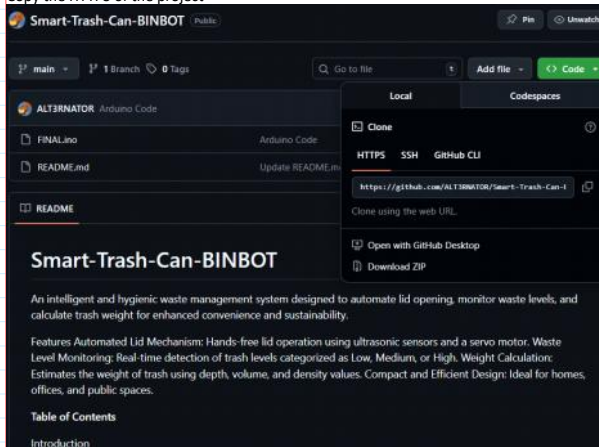
Green File

1-7  39-46

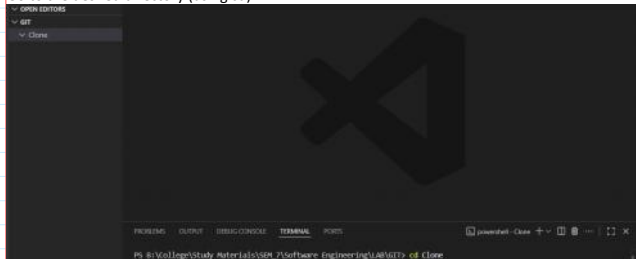
Coning

24 September 2025 10:08 PM

Copy the HTTPS of the project



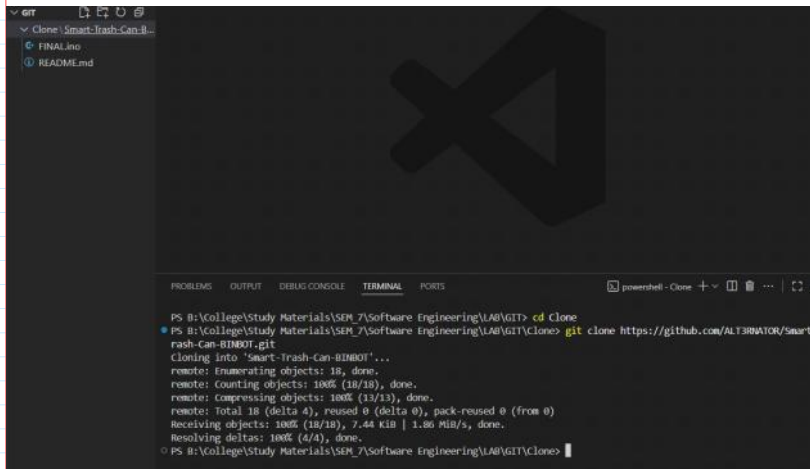
Go to the desired directory (using cd)



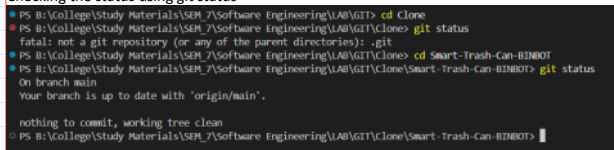
Coning

git clone <https://github.com/ALT3RNATOR/Smart-Trash-Can-BINBOT.git>

Inside clone folder new files appear which are the clone of the repository of which link was provided.



Checking the status using git status



Performing a change on local machine

```

1 # Smart-Trash-Can-BINBOT
69 Weight Calculation:
70
71
72 Calculates trash weight using the measured depth, volume of the bin, and a predefined density constant.
73
74
75 A Test Change
  
```

```

PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT> git --version
git version 2.51.0.windows.1
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT> cd Clone\Smart-Trash-Can-BINBOT
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\Clone\Smart-Trash-Can-BINBOT> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\Clone\Smart-Trash-Can-BINBOT>
  
```

```

PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT> cd Clone\Smart-Trash-Can-BINBOT
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\Clone\Smart-Trash-Can-BINBOT> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\Clone\Smart-Trash-Can-BINBOT> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md
no changes added to commit (use "git add" and/or "git commit -a")
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\Clone\Smart-Trash-Can-BINBOT>
  
```

Handwritten notes:

- Change Performed (pointing to README.md)
- When no changes are performed (pointing to "nothing to commit")
- When changes are performed (pointing to "modified: README.md")
- Indicating what change is performed on which file. (pointing to "modified: README.md")

Creating New File

Create New File "Change_File.py"

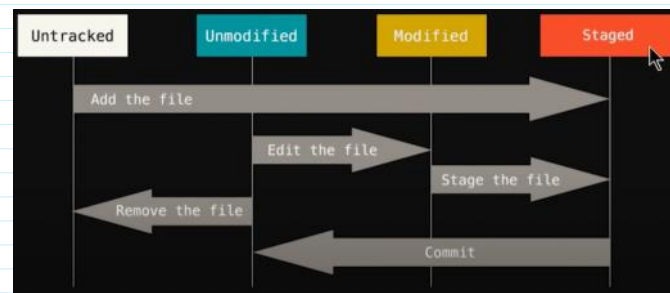
```

PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\Clone\Smart-Trash-Can-BINBOT> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Change_File.py
no changes added to commit (use "git add" and/or "git commit -a")
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\Clone\Smart-Trash-Can-BINBOT>
  
```

Handwritten notes:

- VS code marks a new file as Untracked file. (pointing to Change_File.py in Explorer)
- Modified: README.md → Means some changes are made in this file (pointing to "modified: README.md")
- Change_File.py → Means a new file is added. (pointing to "Untracked files: Change_File.py")



Untracked: the new files that git doesn't yet track.

Modified: changes to the file.

Staged: file is ready to be committed.

Unmodified: Unchanged

Adding the newly created file (STAGED)

git add <file_name>

```

PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\Clone\Smart-Trash-Can-BINBOT> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   Change_File.py
no changes added to commit (use "git add" and/or "git commit -a")
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\Clone\Smart-Trash-Can-BINBOT>
  
```

Handwritten note:

- Using this command the new file / modified file is staged (pointing to "new file: Change_File.py")

```
Change_File.py A
FINALino
README.md M

PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\Clone\Smart-Trash-Can-BINBOT> git add Change_File.py
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\Clone\Smart-Trash-Can-BINBOT> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   Change_File.py -> Changed from untracked to staged.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   README.md

PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\Clone\Smart-Trash-Can-BINBOT>
```

```
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\Clone\Smart-Trash-Can-BINBOT> git add .
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\Clone\Smart-Trash-Can-BINBOT> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   Change_File.py
    modified:   README.md

PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\Clone\Smart-Trash-Can-BINBOT>
```

Finalizing the changes (COMMIT)

git commit -m "messages"

```
EXPLORE
OPEN EDITORS
  README.md Clone...
  Change_File.py Clone...
git
  Clone\Smart-Trash-Can-B...
    Change_File.py
    FINALino
    README.md

Clone > Smart-Trash-Can-BINBOT > Change_File.py
1 print("This is a test file to perform changes on local machine")

PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\Clone\Smart-Trash-Can-BINBOT> git commit -m "Added new file and made changes in the README file"
[main ab9ed99] Added new file and made changes in the README file
2 files changed, 2 insertions(+)
create mode 100644 Change_File.py
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\Clone\Smart-Trash-Can-BINBOT>
```

```
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\Clone\Smart-Trash-Can-BINBOT> git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

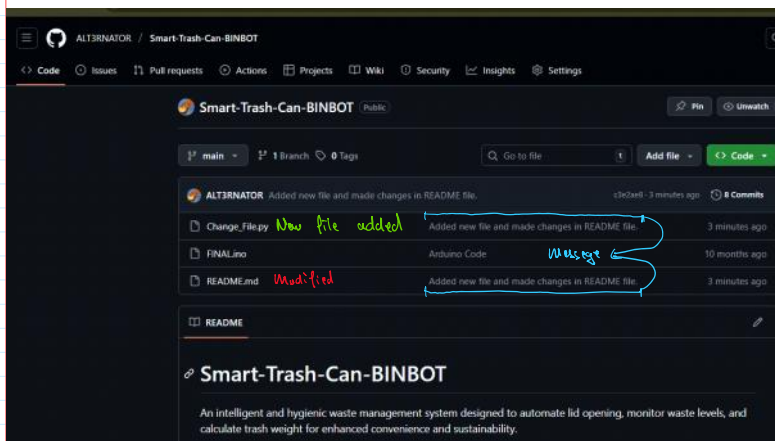
nothing to commit, working tree clean
```

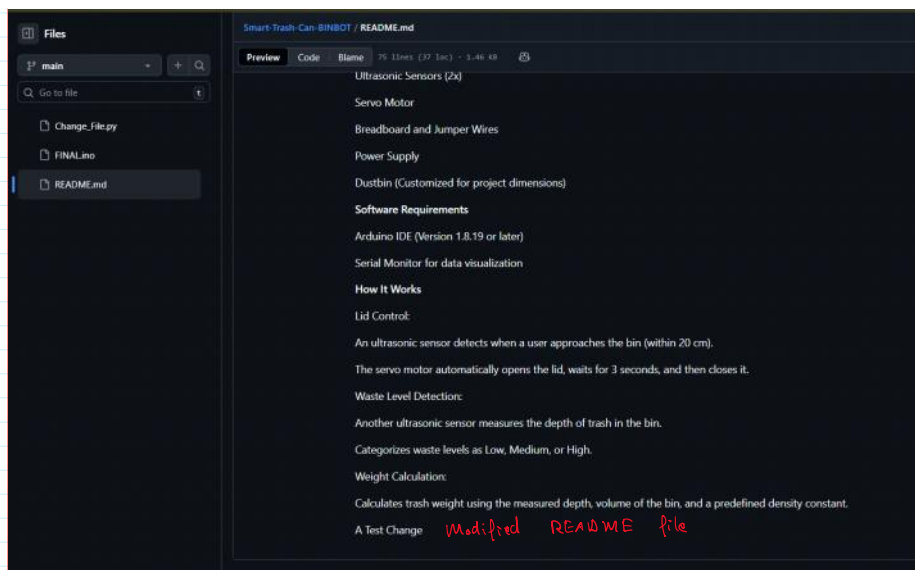
Making changes in the main repo (PUSH)

git push origin main

```
nothing to commit, working tree clean
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\Clone\Smart-Trash-Can-BINBOT> git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 404 bytes | 404.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
to https://github.com/ALT3RNATOR/Smart-Trash-Can-BINBOT.git
ab9ed99..c3e2ae8 main -> main
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\Clone\Smart-Trash-Can-BINBOT> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\Clone\Smart-Trash-Can-BINBOT>
```





New Repo

25 September 2025 10:14 PM

Init

Make new Directory

mkdir <name>

```
nothing to commit, working tree clean
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\Clone\Smart-Trash-Can-BINBOT> cd ..
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\Clone> cd ..
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT> mkdir LocalRepo

Directory: B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT

Mode                LastWriteTime         Length Name
----                -
d-----          25-09-2025  10:27 PM             LocalRepo

PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT> cd LocalRepo
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\LocalRepo> |
```

Checking folder content

It is empty

```
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\LocalRepo> ls
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\LocalRepo> |
```

For checking hidden files

```
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\LocalRepo> ls -force
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\LocalRepo> |
```

Creating git repository

git init

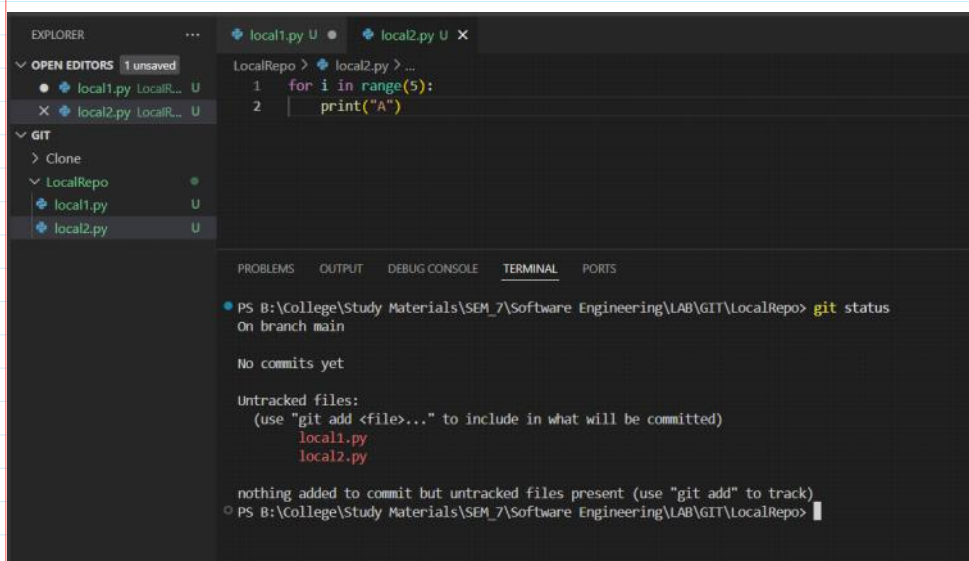
```
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\LocalRepo> ls -force
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\LocalRepo> git init
Initialized empty Git repository in B:/college/Study Materials/SEM_7/Software Engineering/LAB/GIT/LocalRepo/.git/
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\LocalRepo> ls -force

Directory: B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\LocalRepo

Mode                LastWriteTime         Length Name
----                -
d--h--          26-09-2025  12:43 AM             .git

PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\LocalRepo> |
```

Add files to the local repo



Perform actions till COMMIT

git add .

```
nothing added to commit but untracked files present (use "git add" to track)
PS B:\College\Study Materials\SEM 7\Software Engineering\LAB\GIT\LocalRepo> git add .
PS B:\College\Study Materials\SEM 7\Software Engineering\LAB\GIT\LocalRepo> git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   local1.py
        new file:   local2.py

PS B:\College\Study Materials\SEM 7\Software Engineering\LAB\GIT\LocalRepo>
```

git commit -m "msg"

```
PS B:\College\Study Materials\SEM 7\Software Engineering\LAB\GIT\LocalRepo> git commit -m "Added initial files from the local repo."
[main (root-commit) 7eaa72a] Added initial files from the local repo.
 2 files changed, 2 insertions(+)
 create mode 100644 local1.py
 create mode 100644 local2.py
PS B:\College\Study Materials\SEM 7\Software Engineering\LAB\GIT\LocalRepo> git status
On branch main
nothing to commit, working tree clean
PS B:\College\Study Materials\SEM 7\Software Engineering\LAB\GIT\LocalRepo>
```

Now upload it on GITHUB

Create a new repo (on which local repo will be pushed) on github

The screenshot shows the GitHub interface for a newly created repository named 'LocalRepo'. The repository is public and has 0 stars, 0 forks, and 0 watchers. A green handwritten note with an arrow pointing to the repository name says 'New repo created on github to push the local Repo'. Below the repository name, there are sections for 'Set up GitHub Copilot' and 'Add collaborators to this repository'. A 'Quick setup' section provides instructions for setting up the repository on a desktop or via command line. The command line instructions are: `echo "# LocalRepo" >> README.md`, `git init`, `git add README.md`, `git commit -m "first commit"`, `git branch -M main`, `git remote add origin https://github.com/ALT3RNATOR/LocalRepo.git`, and `git push -u origin main`. A red circle highlights the URL `https://github.com/ALT3RNATOR/LocalRepo.git` in the command line instructions.

Add new git remote repo

git remote add origin <link>

git remote add <name of remote repo> <link>

```
PS B:\College\Study Materials\SEM 7\Software Engineering\LAB\GIT\LocalRepo> git remote add origin https://github.com/ALT3RNATOR/LocalRepo.git
PS B:\College\Study Materials\SEM 7\Software Engineering\LAB\GIT\LocalRepo> git remote -v
origin https://github.com/ALT3RNATOR/LocalRepo.git (fetch)
origin https://github.com/ALT3RNATOR/LocalRepo.git (push)
PS B:\College\Study Materials\SEM 7\Software Engineering\LAB\GIT\LocalRepo>
```

A green handwritten note with an arrow pointing to the output of the `git remote -v` command says 'this verifies the repo'.

To check the branch

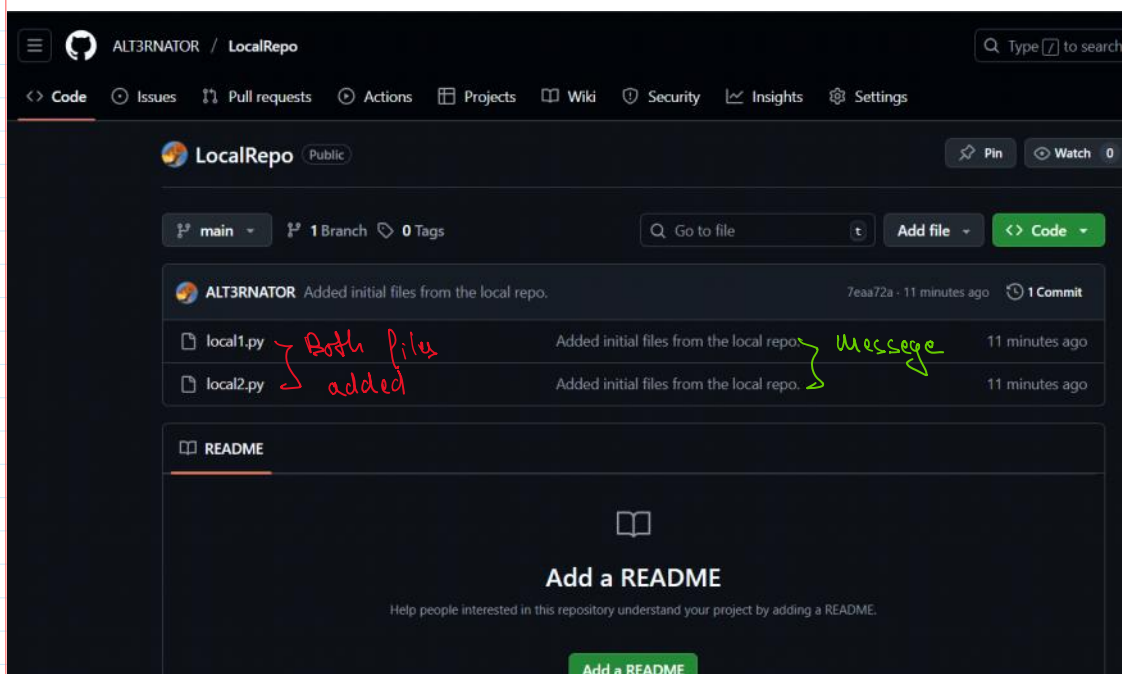
git branch

git branch -M <rename of branch>

Making changes in main repo of github (PUSH)

```
OPEN EDITORS 1 unsaved
  local1.py LocalRepo
  local2.py LocalRepo
GIT
  Clone
  LocalRepo
  local1.py
  local2.py
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\LocalRepo> git remote add origin https://github.com/ALT3RNATOR/LocalRepo.git
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\LocalRepo> git remote -v
origin https://github.com/ALT3RNATOR/LocalRepo.git (fetch)
origin https://github.com/ALT3RNATOR/LocalRepo.git (push)
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\LocalRepo> git branch
* main
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\LocalRepo> git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (4/4), 295 bytes | 147.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/ALT3RNATOR/LocalRepo.git
 * [new branch] main -> main
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\LocalRepo>

git push -u origin main
this makes sure that all
files are pushed to origin main
even if 'origin main' is not mentioned.
```



Adding README.md

```
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\LocalRepo> git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    README.md

nothing added to commit but untracked files present (use "git add" to track)
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\LocalRepo> git add .
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\LocalRepo> git commit -m "Added README files to the repo"
[main 09c1b52] Added README files to the repo
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\LocalRepo> git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 335 bytes | 335.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/ALT3RNATOR/LocalRepo.git
 7eaa72a..09c1b52 main -> main
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\LocalRepo> git status
On branch main
nothing to commit, working tree clean
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\LocalRepo>
```

ALT3RNATOR / LocalRepo

Q Type [Z] to search

<> Code

Issues

Pull requests

Actions

Projects

Wiki

Security

Insights

Settings

LocalRepo

Public

Pin

Watch 0

main

1 Branch

0 Tags

Go to file

Add file

<> Code

ALT3RNATOR

Added README files to the repo

09c1b52 · now

2 Commits

README.md

Added README files to the repo

now

local1.py

Added initial files from the local repo.

15 minutes ago

local2.py

Added initial files from the local repo.

15 minutes ago

README

Readme added

This is Local Repo

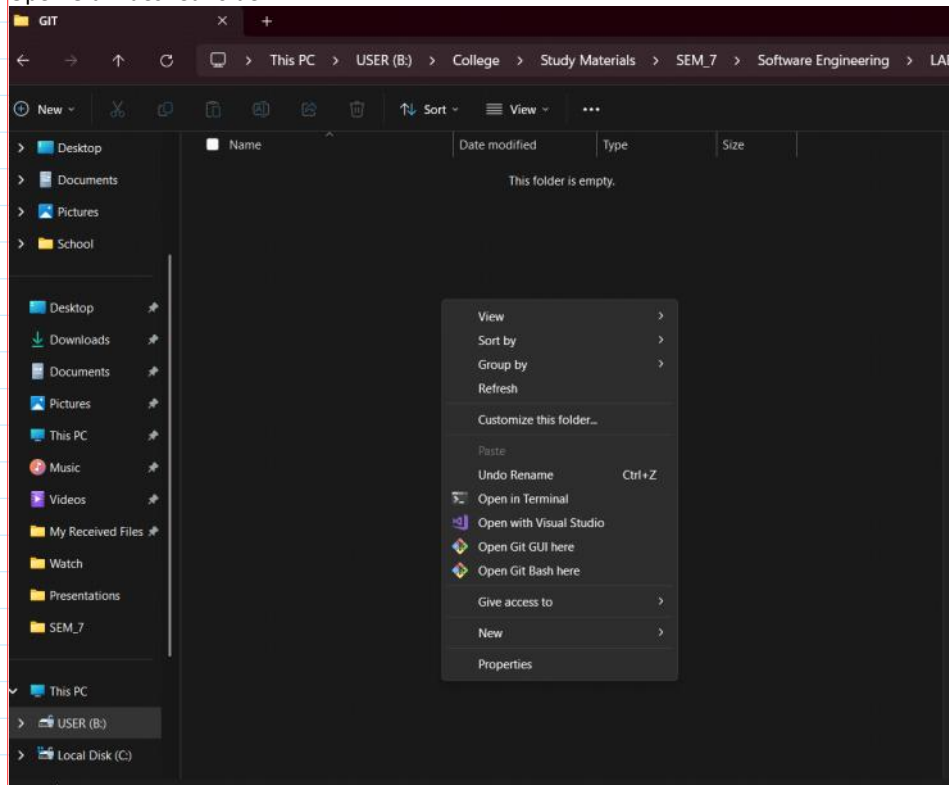
EXP1

24 September 2025 09:24 PM

AIM: Version Control with Git: Practice basic Git operations such as repository creation, file addition, committing changes, and pushing to a remote repository.

STEP 1: Setup

Open Git in desired folder



Configure id and user name

```
ankit@ANKIT_PAVILION MINGW64 /b/College/Study Materials/SEM_7/S
oftware Engineering/LAB/GIT
$ git config --global user.name Ankit Singh

ankit@ANKIT_PAVILION MINGW64 /b/College/Study Materials/SEM_7/S
oftware Engineering/LAB/GIT
$ git config --global user.email ankits4000@gmail.com

ankit@ANKIT_PAVILION MINGW64 /b/College/Study Materials/SEM_7/S
oftware Engineering/LAB/GIT
$ git config --global user.email
ankits4000@gmail.com

ankit@ANKIT_PAVILION MINGW64 /b/College/Study Materials/SEM_7/S
oftware Engineering/LAB/GIT
$ git config --global user.name
Ankit

ankit@ANKIT_PAVILION MINGW64 /b/College/Study Materials/SEM_7/S
oftware Engineering/LAB/GIT
$ |
```

Checking configuration

```

ankit@ANKIT_PAVILION MINGW64 /b/College/Study Materials/SEM_7/S
oftware Engineering/LAB/GIT
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=openssl
http.sslcainfo=C:/Program Files/Git/mingw64/etc/ssl/certs/ca-bundle.crt
core.autocrlf=true
core.fscache=true
core.symlinks=true
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=main
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
user.name=ALT3RNATOR
user.email=ankits4000@gmail.com
user.name=Ankit
user.name=Ankit

```

Open VS code

```

ankit@ANKIT_PAVILION MINGW64 /b/College/Study Materials/SEM_7/Softw
Engineering/LAB/GIT
$ code .

```

On VS Code terminal

```

PS B:\College\Study Materials\SEM_7\Software Engineering\LAB
\GIT> git --version

```

```

PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT> git --version
git version 2.51.0.windows.1

```

STEP 2 : Create Project Directory and navigate into it

```

• PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP1\SE_LAB> mkdir SE_EXP1

Directory: B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP1\SE_LAB

Mode                LastWriteTime         Length Name
----                -
d-----          26-09-2025   03.03 AM                SE_EXP1

• PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP1\SE_LAB> cd SE_EXP1
• PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP1\SE_LAB\SE_EXP1>

```

STEP 3: Initializing git repo

```

• PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP1\SE_LAB\SE_EXP1> git init
Initialized empty Git repository in B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP1\SE_LAB\SE_EXP1/.git/

```

STEP 4: Create new files

Creating entire python file using '@' '@ Block

```

PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP1\SE_LAB\SE_EXP1> @'
>> # Simple Calculator
>>
>> # Take input from the user
>> num1 = float(input("Enter first number: "))
>> operator = input("Enter operator (+, -, *, /): ")
>> num2 = float(input("Enter second number: "))
>>
>> # Perform calculation
>> if operator == '+':
>>     result = num1 + num2
>> elif operator == '-':
>>     result = num1 - num2
>> elif operator == '*':
>>     result = num1 * num2
>> elif operator == '/':
>>     if num2 != 0:
>>         result = num1 / num2
>>     else:
>>         result = "Error! Division by zero."
>> else:
>>     result = "Invalid operator."
>>
>> # Display result
>> print("Result:", result)
>> '@ | Out-File -Encoding utf8 calculator.py
>>

```

To run: python calculator.py

```

PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP1\SE_LAB\SE_EXP1> python calculator.py
Enter first number: 5
Enter operator (+, -, *, /): +
Enter second number: 2
Result: 7.0

```

Adding README.md using **echo**

```

PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP1\SE_LAB\SE_EXP1> echo "# Simple Calculator
- A Python script to perform basic arithmetic operations." > README.md
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP1\SE_LAB\SE_EXP1>

```

Files in the directory

```

PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP1\SE_LAB\SE_EXP1> ls

Directory: B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP1\SE_LAB\SE_EXP1

Mode                LastWriteTime         Length Name
----                -
-a----          26-09-2025   03.04 AM             558 calculator.py
-a----          26-09-2025   03.06 AM             160 README.md

```

STEP 5: Checking the status

git status

```

PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP1\SE_LAB\SE_EXP1> git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    README.md
    calculator.py

nothing added to commit but untracked files present (use "git add" to track)
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP1\SE_LAB\SE_EXP1>

```

STEP 6: Stage the files

git add <file name>

git add .

```

nothing added to commit but untracked files present (use "git add" to track)
• PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP1\SE_LAB\SE_EXP1> git add .
warning: in the working copy of 'calculator.py', LF will be replaced by CRLF the next time Git touches it
• PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP1\SE_LAB\SE_EXP1> git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   README.md
        new file:   calculator.py

```

STEP 7: Commit the files

git commit -m "message"

```

• PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP1\SE_LAB\SE_EXP1> git commit -m "Created a new repo for EXP1 and added python and README files."
[main (root-commit) dcfbdfe] Created a new repo for EXP1 and added python and README files.
 2 files changed, 24 insertions(+)
 create mode 100644 README.md
 create mode 100644 calculator.py
• PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP1\SE_LAB\SE_EXP1>

```

STEP 8: Create a new branch

git branch -M <name>

git branch -M main

```

• PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP1\SE_LAB\SE_EXP1> git branch -M main
• PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP1\SE_LAB\SE_EXP1> git branch
* main
• PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP1\SE_LAB\SE_EXP1>

```

STEP 9: Create a repo on GitHub

The screenshot shows the GitHub interface for a repository named 'Software-Engineer-Lab'. The repository is public and has 0 stars, 0 forks, and 0 watchers. The main content area includes a 'Quick setup' section with instructions for setting up the repository on a desktop or via command line. The command line instructions are as follows:

```

echo "# Software-Engineer-Lab" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/ALT3RNATOR/Software-Engineer-Lab.git
git push -u origin main

```

STEP 10: Add new git remote repo

git remote add origin <link>

git remote add <name of remote repo> <link>

git remote add origin <https://github.com/ALT3RNATOR/Software-Engineer-Lab.git>

```

PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP1\SE_LAB\SE_EXP1> git remote add origin https://github.com/ALT3RNATOR/Software-Engineer-Lab.git
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP1\SE_LAB\SE_EXP1> git status
On branch main
nothing to commit, working tree clean
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP1\SE_LAB\SE_EXP1>

```

STEP 11: Push to GitHub

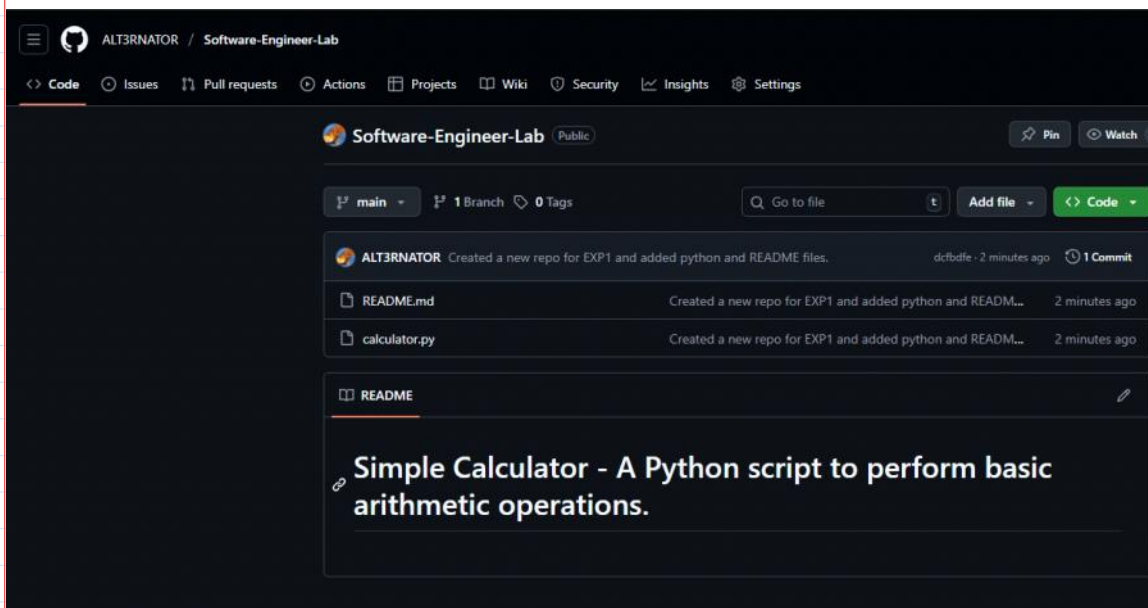
git push -u origin main

```

nothing to commit, working tree clean
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP1\SE_LAB\SE_EXP1> git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 664 bytes | 332.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/ALT3RNATOR/Software-Engineer-Lab.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP1\SE_LAB\SE_EXP1> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP1\SE_LAB\SE_EXP1>

```



EXP2

26 September 2025 03:37 AM

AIM: Unit Testing with pytest: Write and execute unit tests for Python functions using the pytest framework, covering different test cases and assertions.

STEP 1: Installing required Tools and verification

```
python --version
```

```
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP2> python --version  
Python 3.13.7
```

```
pip install pytest
```

```
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location  
Successfully installed colorama-0.4.6 iniconfig-2.1.0 packaging-25.0 pluggy-1.6.0 pygments-2.19.2 pytest-8.4.2
```

STEP2 : Create a Python module to test

```
mkdir EXP2
```

```
cd EXP2
```

```
@'
```

```
def add(a, b):  
    return a + b
```

```
def divide(a, b):  
    if b == 0:  
        raise ValueError("Cannot divide by zero")  
    return a / b
```

```
def is_even(n):  
    return n % 2 == 0
```

```
'@ | Out-File -Encoding UTF8 math_utils.py
```

```
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP2> @'  
>> def add(a, b):  
>>     return a + b  
>>  
>> def divide(a, b):  
>>     if b == 0:  
>>         raise ValueError("Cannot divide by zero")  
>>     return a / b  
>>  
>> def is_even(n):  
>>     return n % 2 == 0  
>> '@ | Out-File -Encoding UTF8 math_utils.py  
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP2> |
```

Explanation:

- '@' ... '@' → multi-line string (here-string)
- Out-File -Encoding UTF8 → writes the string to a file

STEP 3: Create Test file

```
@'
```

```
import pytest  
from math_utils import add, divide, is_even
```

```
def test_add_numbers():  
    assert add(2, 3) == 5  
    assert add(-1, 1) == 0  
    assert add(0, 0) == 0
```

```
def test_divide_numbers():  
    assert divide(10, 2) == 5  
    assert divide(-9, 3) == -3
```

```
def test_divide_by_zero():  
    with pytest.raises(ValueError):  
        divide(5, 0)
```

```
@pytest.mark.parametrize("num,expected", [
    (2, True),
    (3, False),
    (0, True),
    (-4, True),
])
def test_is_even(num, expected):
    assert is_even(num) == expected
'@ | Out-File -Encoding UTF8 test_math_utils.py
```

```
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP2> @'
>> import pytest
>> from math_utils import add, divide, is_even
>>
>> def test_add_numbers():
>>     assert add(2, 3) == 5
>>     assert add(-1, 1) == 0
>>     assert add(0, 0) == 0
>>
>> def test_divide_numbers():
>>     assert divide(10, 2) == 5
>>     assert divide(-9, 3) == -3
>>
>> def test_divide_by_zero():
>>     with pytest.raises(ValueError):
>>         divide(5, 0)
>>
>> @pytest.mark.parametrize("num,expected", [
>>     (2, True),
>>     (3, False),
>>     (0, True),
>>     (-4, True),
>> ])
>> def test_is_even(num, expected):
>>     assert is_even(num) == expected
>> '@ | Out-File -Encoding UTF8 test_math_utils.py
>>
```

STEP 4: Run pytest

python -m pytest -v

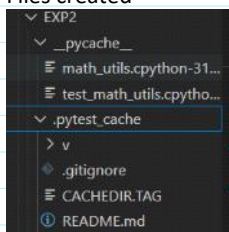
```
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP2> python -m pytest -v
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-8.4.2, pluggy-1.6.0 -- C:\Python313\python.exe
cachedir: .pytest_cache
rootdir: B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LAB\EXP2
collected 7 items

test_math_utils.py::test_add_numbers PASSED [ 14%]
test_math_utils.py::test_divide_numbers PASSED [ 28%]
test_math_utils.py::test_divide_by_zero PASSED [ 42%]
test_math_utils.py::test_is_even[2-True] PASSED [ 57%]
test_math_utils.py::test_is_even[3-False] PASSED [ 71%]
test_math_utils.py::test_is_even[0-True] PASSED [ 85%]
test_math_utils.py::test_is_even[-4-True] PASSED [100%]

===== 7 passed in 0.02s =====
```

Output:

Files created



7 test collected and 7 test passed

Result:

Unit testing performed successfully, functions add(), divide() and is_even() verified

EXP3

26 September 2025 03:56 AM

AIM: To set up a basic Continuous Integration (CI) pipeline using GitHub Actions that automatically builds and tests code on every commit.

STEP 1: Create workflow folder in local repo

```
mkdir .github
mkdir .github\workflows
```

```
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS> mkdir .github
>> mkdir .github\workflows
>>

Directory: B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS

Mode                LastWriteTime         Length Name
----                -
d-----         26-09-2025   09:55 AM             .github

Directory: B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS\.github

Mode                LastWriteTime         Length Name
----                -
d-----         26-09-2025   09:55 AM             workflows

PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS>
```

STEP 2: Create a workflow file

```
@'
name: Python CI - EXP3

# Trigger workflow only on changes inside EXP3
on:
  push:
    paths:
      - "EXP3/**"
    branches: [ "main" ]
  pull_request:
    paths:
      - "EXP3/**"
    branches: [ "main" ]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: "3.x"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install pytest

      - name: Run pytest for EXP3
        run: pytest EXP3/ -v
'@ | Out-File -Encoding UTF8 .github\workflows\exp3-ci.yml
```

```

PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS> @'
>> name: Python CI - EXP3
>>
>> on:
>>   push:
>>     paths:
>>       - "EXP3/**"
>>     branches: [ "main" ]
>>   pull_request:
>>     paths:
>>       - "EXP3/**"
>>     branches: [ "main" ]
>>
>> jobs:
>>   build:
>>     runs-on: ubuntu-latest
>>
>>     steps:
>>     - name: Checkout repository
>>       uses: actions/checkout@v3
>>
>>     - name: Set up Python
>>       uses: actions/setup-python@v4
>>       with:
>>         python-version: "3.x"
>>
>>     - name: Install dependencies
>>       run: |
>>         python -m pip install --upgrade pip
>>         pip install pytest
>>
>>     - name: Run pytest for EXP3
>>       run: pytest EXP3/ -v
>> '@ | Out-File -Encoding UTF8 .github\workflows\exp3-ci.yml

```

STEP 3: Add, Commit, Push the workflow

```

git add .github\workflows\exp3-ci.yml
git commit -m "Add GitHub Actions CI workflow for EXP3"
git push origin main

```

```

PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS> git push origin main
>>
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 665 bytes | 665.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/ALT3RNATOR/Software-Engineering-Lab.git
   6dd2cc3..feb0a9c  main -> main
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS>

```

The screenshot displays the GitHub Actions interface for the repository **ALT3RNATOR / Software-Engineering-Lab**. The **Actions** tab is active, showing a list of workflow runs. The left sidebar contains a navigation menu with options: **All workflows**, **Python CI - EXP3**, **Management**, **Caches**, **Attestations**, **Runners**, **Usage metrics**, and **Performance metrics**. The main area, titled **All workflows**, shows two workflow runs:

Workflow Name	Commit	Branch	Status	Duration	Actor
Python CI - EXP3	08406d9	main	Completed	11 minutes ago	ALT3RNATOR
Triger the workflow	6a24b46	main	Completed	12 minutes ago	ALT3RNATOR

AIM: To identify and refactor Python code to improve quality, readability, and performance by applying techniques such as extracting methods, removing duplication, and improving naming conventions.

STEP 1: Create a refactored code and unit tests

```
@'
# calculator.py - Refactored Calculator

def get_input():
    num1 = float(input("Enter first number: "))
    operator = input("Enter operator (+, -, *, /): ")
    num2 = float(input("Enter second number: "))
    return num1, operator, num2

def calculate(num1, operator, num2):
    if operator == '+':
        return num1 + num2
    elif operator == '-':
        return num1 - num2
    elif operator == '*':
        return num1 * num2
    elif operator == '/':
        return "Error! Division by zero." if num2 == 0 else num1 / num2
    else:
        return "Invalid operator."

def main():
    num1, operator, num2 = get_input()
    result = calculate(num1, operator, num2)
    print("Result:", result)

if __name__ == "__main__":
    main()
'@ | Out-File -Encoding UTF8 EXP4\calculator.py
```

```
@'
# test_calculator.py
import EXP4.calculator as calc

def test_addition():
    assert calc.calculate(2, "+", 3) == 5

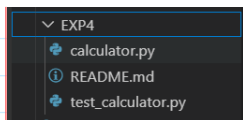
def test_subtraction():
    assert calc.calculate(5, "-", 2) == 3

def test_multiplication():
    assert calc.calculate(3, "*", 4) == 12

def test_division():
    assert calc.calculate(10, "/", 2) == 5

def test_division_by_zero():
    assert calc.calculate(5, "/", 0) == "Error! Division by zero."

def test_invalid_operator():
    assert calc.calculate(5, "%", 2) == "Invalid operator."
'@ | Out-File -Encoding UTF8 EXP4\test_calculator.py
```



STEP 2: Add EXP4 to Git, commit,

```
git add EXP4
```

```
git commit -m "Add EXP4: Refactored Calculator with tests"
```

STEP 3: Run the test on local machine

```
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS> python -m pytest -v EXP4
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-8.4.2, pluggy-1.6.0 -- C:\Python313\python.exe
cachedir: .pytest cache
rootdir: B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS
collected 6 items

EXP4/test_calculator.py::test_addition PASSED [ 16%]
EXP4/test_calculator.py::test_subtraction PASSED [ 33%]
EXP4/test_calculator.py::test_multiplication PASSED [ 50%]
EXP4/test_calculator.py::test_division PASSED [ 66%]
EXP4/test_calculator.py::test_division_by_zero PASSED [ 83%]
EXP4/test_calculator.py::test_invalid_operator PASSED [100%]

===== 6 passed in 0.03s =====
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS>
```

STEP 4: Push the refactored code

```
git add .
```

```
git commit -m "Refactor: extract validation, remove duplication, improve naming"
```

```
git push origin main
```

```
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS> git add .
>> git commit -m "Refactor: extract validation, remove duplication, improve naming"
>> git push origin main
[main 6f4e2bf] Refactor: extract validation, remove duplication, improve naming
2 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 EXP4/__pycache__/calculator.cpython-313.pyc
create mode 100644 EXP4/__pycache__/test_calculator.cpython-313-pytest-8.4.2.pyc
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 8 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 2.65 KiB | 2.65 MiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/ALT3RNATOR/Software-Engineering-Lab.git
17acec8..6f4e2bf main -> main
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS>
```

AIM: Write and document Python functions and classes using docstrings and generate HTML documentation using MkDocs tool.

STEP 1: Create a well-documented code

```
@'
''''
```

shapes.py

Example module demonstrating functions, classes, and docstrings
for Software Engineering Lab EXP5.

```
''''
```

```
from dataclasses import dataclass
from typing import Iterable
```

```
def add(a: float, b: float) -> float:
    """Add two numbers.
```

Args:

a: First number.
b: Second number.

Returns:

The sum of ``a`` and ``b``.

Examples:

```
>>> add(2, 3)
5
''''
```

```
return a + b
```

```
def average(values: Iterable[float]) -> float:
    """Compute the average of a sequence of numbers.
```

Args:

values: Any iterable of numeric values.

Returns:

The arithmetic mean of all values.

Raises:

ValueError: If ``values`` is empty.

Examples:

```
>>> average([1, 2, 3, 4])
2.5
''''
```

```
values = list(values)
```

```
if not values:
```

```
    raise ValueError("values must not be empty")
```

```
return sum(values) / len(values)
```

```
@dataclass
```

```
class Rectangle:
```

```
    """Represent a rectangle shape.
```

Attributes:

width: The width of the rectangle (must be non-negative).
height: The height of the rectangle (must be non-negative).

```
''''
```

```
width: float
```

```
height: float
```

```
def area(self) -> float:
```

```
    """Calculate the area of the rectangle.
```

Returns:

The area as ``width \* height``.

Examples:

```
>>> rect = Rectangle(3, 4)
```

```

    >>> rect.area()
    12
    """
    return self.width * self.height

def perimeter(self) -> float:
    """Calculate the perimeter of the rectangle.

    Returns:
        The perimeter as ``2 * (width + height)``.

    Examples:
        >>> rect = Rectangle(3, 4)
        >>> rect.perimeter()
        14
    """
    return 2 * (self.width + self.height)
'@ | Out-File -Encoding UTF8 EXP5\shapes.py

```

STEP 2: Add a test file

```

@'
# test_shapes.py

from EXP5.shapes import add, average, Rectangle
import pytest

def test_add():
    assert add(2, 3) == 5
    assert add(-1, 1) == 0

def test_average():
    assert average([1, 2, 3, 4]) == 2.5
    with pytest.raises(ValueError):
        average([])

def test_rectangle():
    rect = Rectangle(3, 4)
    assert rect.area() == 12
    assert rect.perimeter() == 14
'@ | Out-File -Encoding UTF8 EXP5\test_shapes.py

```

STEP 3: Run the test

```

Python -m pytest -v EXP5
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS> python -m pytest -v EXP5
===== test session starts =====
platform win32 -- Python 3.14.0, pytest-9.0.1, pluggy-1.6.0 -- C:\Python314\python.exe
cachedir: .pytest_cache
rootdir: B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS
collected 3 items

EXP5/test_shapes.py::test_add PASSED
EXP5/test_shapes.py::test_average PASSED
EXP5/test_shapes.py::test_rectangle PASSED

===== 3 passed in 0.01s =====

```

STEP 4: Create mkdocs.yml

```

@'
site_name: Software Engineering Labs
nav:
  - Home: index.md
  - EXP1: exp1.md
  - EXP2: exp2.md
  - EXP3: exp3.md
  - EXP4: exp4.md
  - EXP5 - Docstrings & MkDocs: exp5.md

theme:
  name: mkdocs
'@ | Out-File -Encoding UTF8 mkdocs.yml

```

STEP 5: Create docs/index.md

```

@'
# Software Engineering Labs

```

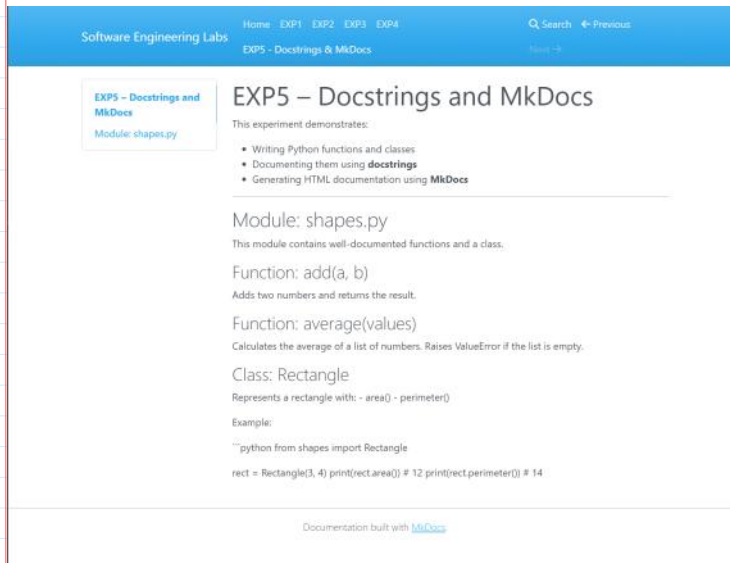
Welcome to the documentation site for your Software Engineering lab experiments.
Use the navigation to explore each experiment.
'@ | Out-File -Encoding UTF8 docs\index.md

STEP 6: Configure mkdocs

pip install mkdocs
mkdocs serve

```
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS> mkdocs serve
INFO - Building documentation...
INFO - Cleaning site directory
WARNING - A reference to 'exp1.md' is included in the 'nav' configuration, which is not found in the documentation files.
WARNING - A reference to 'exp2.md' is included in the 'nav' configuration, which is not found in the documentation files.
WARNING - A reference to 'exp3.md' is included in the 'nav' configuration, which is not found in the documentation files.
WARNING - A reference to 'exp4.md' is included in the 'nav' configuration, which is not found in the documentation files.
INFO - Documentation built in 0.92 seconds
INFO - [23:15:47] Serving on http://127.0.0.1:8000/
```

Open Browser at <http://127.0.0.1:8000/>



To stop server:
CTRL+C

mkdocs build

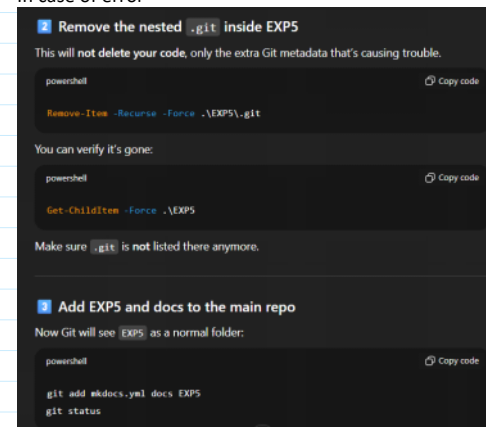
```
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS> mkdocs build
INFO - Cleaning site directory
INFO - Building documentation to directory: B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS\site
WARNING - A reference to 'exp1.md' is included in the 'nav' configuration, which is not found in the documentation files.
WARNING - A reference to 'exp2.md' is included in the 'nav' configuration, which is not found in the documentation files.
WARNING - A reference to 'exp3.md' is included in the 'nav' configuration, which is not found in the documentation files.
WARNING - A reference to 'exp4.md' is included in the 'nav' configuration, which is not found in the documentation files.
INFO - Documentation built in 0.08 seconds
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS>
```

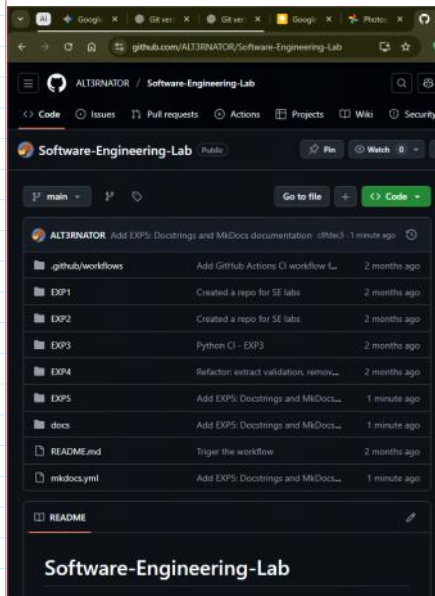
STEP 7: Push to Github

git add mkdocs.yml docs EXP5
git commit -m "Add EXP5: Docstrings and MkDocs documentation"
git push origin main

```
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS> git co
>> git push origin main
>>
[main c8fdac3] Add EXP5: Docstrings and MkDocs documentation
7 files changed, 156 insertions(+)
create mode 100644 EXP5/_pycache_/shapes.cpython-314.pyc
create mode 100644 EXP5/_pycache_/test_shapes.cpython-314-pytest-9.0.1.pyc
create mode 100644 EXP5/shapes.py
create mode 100644 EXP5/test_shapes.py
create mode 100644 docs/exp5.md
create mode 100644 docs/index.md
create mode 100644 mkdocs.yml
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 28 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 5.54 KiB | 2.77 MiB/s, done.
Total 12 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/ALT3RNATOR/Software-Engineering-Lab.git
6f4e2bf..c8fdac3 main -> main
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS>
```

In case of error





EXP6

23 November 2025 09:46 PM

AIM: Write and document Python functions and classes using docstrings and generate HTML documentation using Sphinx tool.

STEP1: Create a folder and package

mkdir EXP6

New-Item EXP6__init__.py -ItemType File

STEP2: Create shapes_sphinx.py with docstrings

```
@'  
''''
```

shapes_sphinx.py

Example module for Ex6 showing functions, classes, and reStructuredText-style docstrings for Sphinx.

```
from dataclasses import dataclass  
from typing import Iterable
```

```
def add(a: float, b: float) -> float:  
    """Add two numbers.
```

```
    :param a: First number.  
    :type a: float  
    :param b: Second number.  
    :type b: float  
    :return: Sum of ``a`` and ``b``.  
    :rtype: float
```

Example::

```
>>> add(2, 3)  
5  
"""  
return a + b
```

```
def average(values: Iterable[float]) -> float:  
    """Compute the arithmetic mean of a sequence of numbers.
```

```
    :param values: Iterable of numeric values.  
    :type values: Iterable[float]  
    :raises ValueError: If ``values`` is empty.  
    :return: Arithmetic mean of the values.  
    :rtype: float  
    """
```

```
    values = list(values)  
    if not values:  
        raise ValueError("values must not be empty")  
    return sum(values) / len(values)
```

```
@dataclass  
class Rectangle:
```

```
    """Represent a rectangle.
```

```
    :param width: Rectangle width.  
    :type width: float  
    :param height: Rectangle height.  
    :type height: float  
    """
```

```
    width: float  
    height: float
```

```
    def area(self) -> float:  
        """Return the area of the rectangle.  
  
        :return: Area = ``width * height``.
```

```

:rtype: float
"""
return self.width * self.height

def perimeter(self) -> float:
    """Return the perimeter of the rectangle.

    :return: Perimeter = ``2 * (width + height)``.
    :rtype: float
    """
    return 2 * (self.width + self.height)
'@ | Out-File -Encoding UTF8 .\EXP6\shapes_sphinx.py

```

STEP3: Install Sphinx

```

python -m pip install --upgrade pip
python -m pip install sphinx

```

STEP4: Create a Sphinx docs project

```
sphinx-quickstart docs_exp6_sphinx
```

You'll get interactive questions. Suggested answers:

- Separate source and build directories (y/n) → **y**
- Project name → **Software Engineering Labs - Ex6**
- Author name → your name
- Project release → **1.0**
- Rest → defaults (just press Enter)

```

PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS> sphinx-quickstart docs_exp6_sphinx
Welcome to the Sphinx 8.2.3 quickstart utility.

Please enter values for the following settings (just press Enter to
accept a default value, if one is given in brackets).

Selected root path: docs_exp6_sphinx

You have two options for placing the build directory for Sphinx output.
Either, you use a directory "_build" within the root path, or you separate
"source" and "build" directories within the root path.
> Separate source and build directories (y/n) [n]: 

```

```

PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS> sphinx-quickstart docs_exp6_sphinx
Welcome to the Sphinx 8.2.3 quickstart utility.

Please enter values for the following settings (just press Enter to
accept a default value, if one is given in brackets).

Selected root path: docs_exp6_sphinx

You have two options for placing the build directory for Sphinx output.
Either, you use a directory "_build" within the root path, or you separate
"source" and "build" directories within the root path.
> Separate source and build directories (y/n) [n]: y

The project name will occur in several places in the built documentation.
> Project name: Software Engineering Labs - EXP6
> Author name(s): Ankit Singh
> Project release []: 1.0

If the documents are to be written in a language other than English,
you can select a language here by its language code. Sphinx will then
translate text that it generates into that language.

For a list of supported codes, see
https://www.sphinx-doc.org/en/master/usage/configuration.html#confval-language.
> Project language [en]:

Creating file B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS\docs_exp6_sphinx\source\conf.py.
Creating file B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS\docs_exp6_sphinx\source\index.rst.
Creating file B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS\docs_exp6_sphinx\Makefile.
Creating file B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS\docs_exp6_sphinx\make.bat.

Finished: An initial directory structure has been created.

You should now populate your master file B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS\docs_exp6_sphinx\source\index.rst and create other documentation
source files. Use the Makefile to build the docs, like so:

    make builder

where "builder" is one of the supported builders, e.g. html, latex or linkcheck.

```

STEP5: Configure Sphinx to document your EXP6 module

Open conf.py:
notepad .\docs_exp6_sphinx\source\conf.py

At the **top of conf.py**, just under the comments, add:

```
import os
import sys
sys.path.insert(0, os.path.abspath('../..'))
```

This makes Python see SE_LABS (where EXP6 lives) on sys.path, so EXP6.shapes_sphinx can be imported.

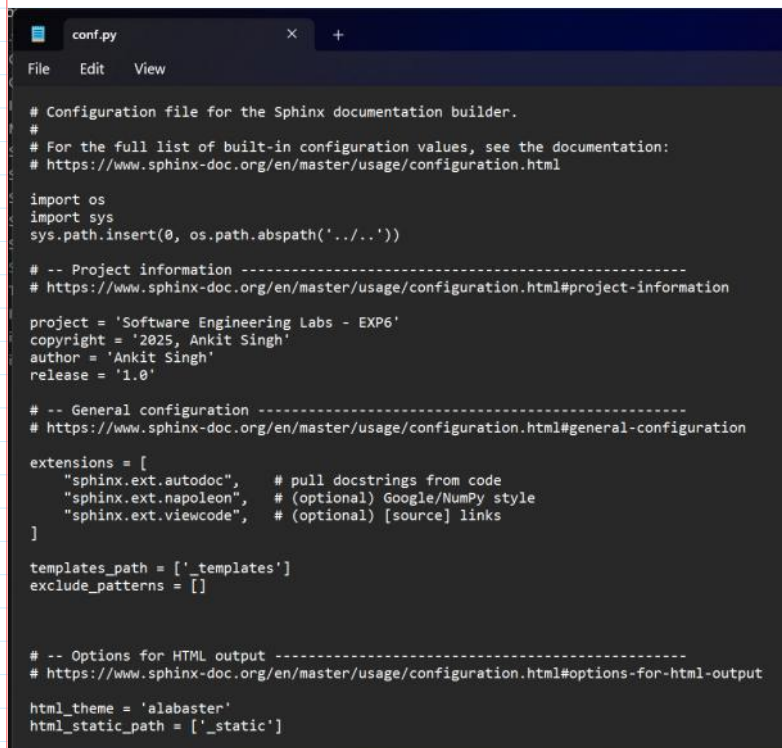
In the same conf.py, find:

```
extensions = []
```

Change it to:

```
extensions = [
    "sphinx.ext.autodoc", # pull docstrings from code
    "sphinx.ext.napoleon", # (optional) Google/NumPy style
    "sphinx.ext.viewcode", # (optional) [source] links
]
```

Save and close Notepad.



```
conf.py
File Edit View

# Configuration file for the Sphinx documentation builder.
#
# For the full list of built-in configuration values, see the documentation:
# https://www.sphinx-doc.org/en/master/usage/configuration.html

import os
import sys
sys.path.insert(0, os.path.abspath('../..'))

# -- Project information -----
# https://www.sphinx-doc.org/en/master/usage/configuration.html#project-information

project = 'Software Engineering Labs - EXP6'
copyright = '2025, Ankit Singh'
author = 'Ankit Singh'
release = '1.0'

# -- General configuration -----
# https://www.sphinx-doc.org/en/master/usage/configuration.html#general-configuration

extensions = [
    "sphinx.ext.autodoc", # pull docstrings from code
    "sphinx.ext.napoleon", # (optional) Google/NumPy style
    "sphinx.ext.viewcode", # (optional) [source] links
]

templates_path = ['_templates']
exclude_patterns = []

# -- Options for HTML output -----
# https://www.sphinx-doc.org/en/master/usage/configuration.html#options-for-html-output

html_theme = 'alabaster'
html_static_path = ['_static']
```

STEP6: Create a page that uses autodoc on your module

@'

Ex6 – Sphinx Documentation

=====

This page documents the module ``EXP6.shapes\_sphinx`` using Sphinx autodoc.

```
.. automodule:: EXP6.shapes_sphinx
```

```
:members:
```

```
:undoc-members:
```

```
:show-inheritance:
```

```
'@ | Out-File -Encoding UTF8 .\docs_exp6_sphinx\source\exp6.rst
```

Open index.rst:

```
notepad .\docs_epx6_sphinx\source\index.rst
```

Replace contents with:

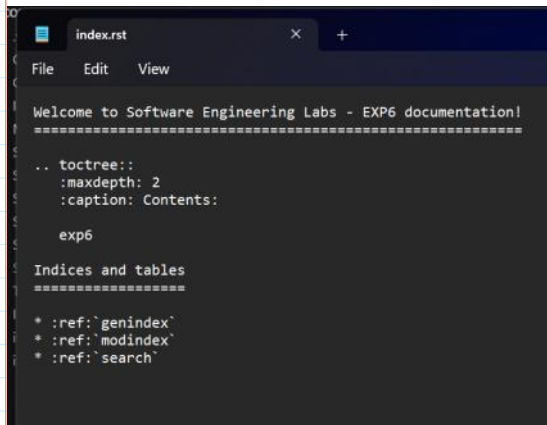
Welcome to Software Engineering Labs - EXP6 documentation!

=====

==

```
.. toctree::
```

```
:maxdepth: 2
:caption: Contents:
exp6
Indices and tables
=====
* :ref:`genindex`
* :ref:`modindex`
* :ref:`search`
Save and close.
```



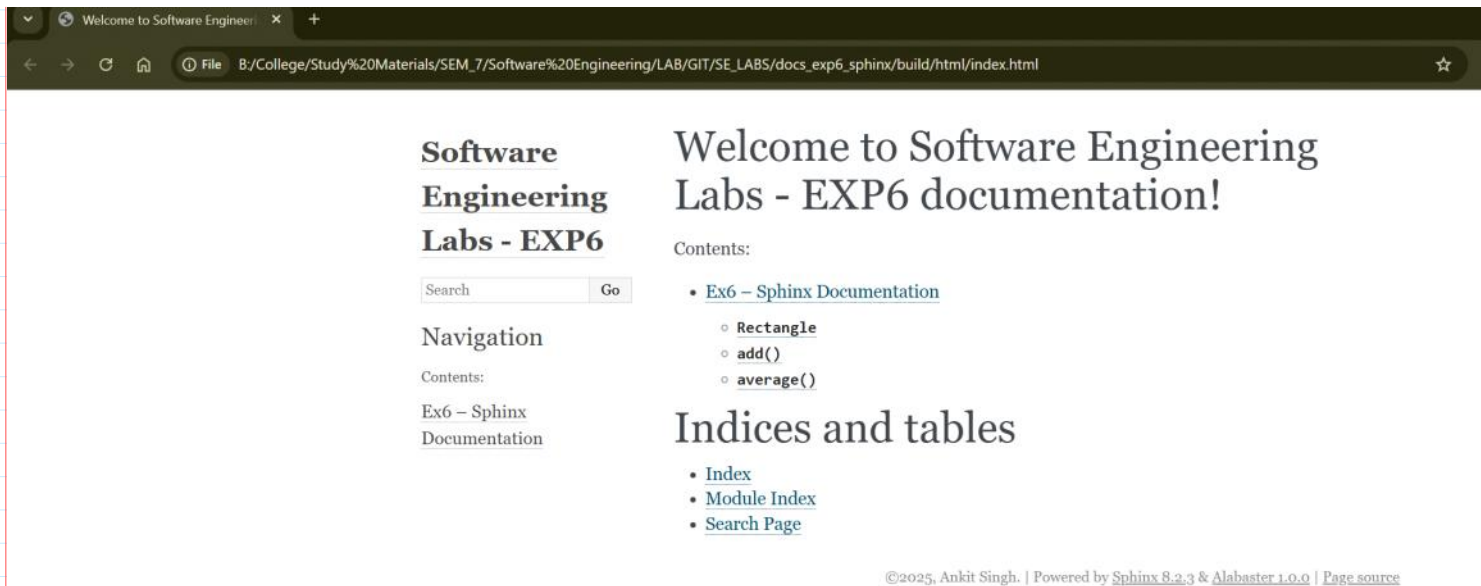
STEP7: Build HTML documentation

```
cd .\docs_exp6_sphinx
.\make.bat html
```

```
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS> cd "B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS\docs_exp6_sphinx"
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS\docs_exp6_sphinx> .\make.bat html
Running Sphinx v8.2.3
loading translations [en]... done
making output directory... done
building [mo]: targets for 0 po files that are out of date
writing output...
building [html]: targets for 2 source files that are out of date
updating environment: [new config] 2 added, 0 changed, 0 removed
reading sources... [100%] index
looking for now-outdated files... none found
pickling environment... done
checking consistency... done
preparing documents... done
copying assets...
copying static files...
Writing evaluated template result to B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS\docs_exp6_sphinx\build\html\_static\basic.css
Writing evaluated template result to B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS\docs_exp6_sphinx\build\html\_static\documentation_options.js
Writing evaluated template result to B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS\docs_exp6_sphinx\build\html\_static\language_data.js
Writing evaluated template result to B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS\docs_exp6_sphinx\build\html\_static\alabaster.css
copying static files: done
copying extra files...
copying extra files: done
copying assets: done
writing output... [100%] index
generating indices... genindex py-modindex done
highlighting module code... [100%] EXP6.shapes_sphinx
writing additional pages... search done
dumping search index in English (code: en)... done
dumping object inventory... done
build succeeded.

The HTML pages are in build/html.
PS B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS\docs_exp6_sphinx> start .\build\html\index.html
```

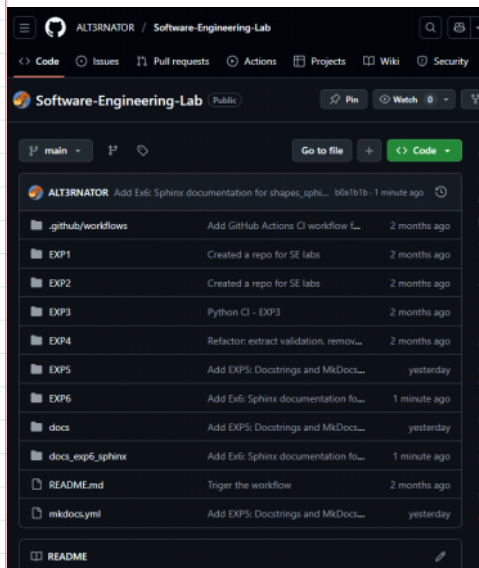
```
start .\build\html\index.html
```



STEP8: Push to Git

```
cd "B:\College\Study Materials\SEM_7\Software Engineering\LAB\GIT\SE_LABS"
```

```
git add EXP6 docs_exp6_sphinx
git commit -m "Add Ex6: Sphinx documentation for shapes_sphinx module"
git push origin main
```



EXP7

24 November 2025 02:49 AM

AIM: Implement mocking and dependency injection techniques in python using libraries unittest.mock or dependency-injector.

STEP1: Create the real code with dependency injection

We'll simulate sending emails:

- EmailClient – a dependency (would talk to SMTP / API in real life)
- UserNotifier – depends on EmailClient (passed in = **dependency injection**)

Create EXP7\nnotification.py:

```
@'
''''
```

notification.py

Example module for EXP7 showing:

- dependency injection
 - mocking with unittest.mock
- ```
''''
```

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class EmailClient:
```

```
 """Simple email client dependency.
```

```
 In a real app, this would talk to an SMTP server or external API.
```

```
 Here we only simulate its interface for testing with mocks.
```

```
 ''''
```

```
 sender_address: str = "no-reply@example.com"
```

```
 def send_email(self, to: str, subject: str, body: str) -> None:
```

```
 """Send an email.
```

```
 In production code this would send a real email.
```

```
 For testing, we will **mock** this method.
```

```
 ''''
```

```
 print(f"Sending email to {to}: {subject}\n{body}")
```

```
class UserNotifier:
```

```
 """Service that sends emails to users.
```

```
 Uses dependency injection: an EmailClient instance is passed
```

```
 into the constructor instead of being created inside.
```

```
 ''''
```

```
 def __init__(self, email_client: EmailClient) -> None:
```

```
 self.email_client = email_client
```

```
 def send_welcome_email(self, user_email: str) -> None:
```

```
 """Send a welcome email to a new user."""
```

```
 subject = "Welcome!"
```

```
body = "Thanks for signing up."
self.email_client.send_email(user_email, subject, body)
```

```
def send_password_reset(self, user_email: str, reset_link: str) ->
```

None:

```
"""Send password reset email."""
subject = "Password Reset"
body = f"Click here to reset your password: {reset_link}"
self.email_client.send_email(user_email, subject, body)
```

```
def send_system_announcement(to_email: str, message: str) ->
```

None:

```
"""Function-style API that creates its own EmailClient internally.
```

```
We'll use unittest.mock.patch to replace EmailClient here in tests.
"""
```

```
client = EmailClient()
client.send_email(to_email, "System Announcement", message)
```

```
@ | Out-File -Encoding UTF8 EXP7\notification.py
```

Key points:

- **Dependency Injection:** UserNotifier receives email\_client in \_\_init\_\_.
- This makes it easy to pass in a **mock** in tests.

## STEP2: Create test using unittest + unittest.mock

We'll demonstrate:

- MagicMock as a fake dependency
- side\_effect to simulate errors
- patch to replace EmailClient in send\_system\_announcement

Create EXP7\test\_notification.py:

```
@'
```

```
import unittest
```

```
from unittest.mock import MagicMock, patch
```

```
from EXP7.notification import UserNotifier, EmailClient,
send_system_announcement
```

```
class TestUserNotifier(unittest.TestCase):
```

```
def test_send_welcome_email_uses_email_client(self):
```

```
 # Arrange: create a mock EmailClient
```

```
 mock_client = MagicMock(spec=EmailClient)
```

```
 notifier = UserNotifier(mock_client)
```

```
 # Act
```

```
 notifier.send_welcome_email("user@example.com")
```

```
 # Assert: ensure send_email was called with correct arguments
```

```
 mock_client.send_email.assert_called_once_with(
```

```
 "user@example.com",
```

```
 "Welcome!",
```

```
 "Thanks for signing up.",
```

```
)
```

```

def test_send_password_reset_with_link(self):
 mock_client = MagicMock(spec=EmailClient)
 notifier = UserNotifier(mock_client)

 reset_link = "https://example.com/reset?token=abc123"
 notifier.send_password_reset("user@example.com", reset_link)

 mock_client.send_email.assert_called_once_with(
 "user@example.com",
 "Password Reset",
 f"Click here to reset your password: {reset_link}",
)

def test_email_client_raises_error(self):
 # Simulate failure in dependency using side_effect
 mock_client = MagicMock(spec=EmailClient)
 mock_client.send_email.side_effect = RuntimeError("SMTP
down")

 notifier = UserNotifier(mock_client)

 with self.assertRaises(RuntimeError):
 notifier.send_welcome_email("user@example.com")

class TestSystemAnnouncement(unittest.TestCase):
 @patch("EXP7.notification.EmailClient")
 def test_send_system_announcement_uses_email_client(self,
MockEmailClient):
 # Mock the EmailClient class itself
 instance = MockEmailClient.return_value # this is the fake
instance

 send_system_announcement("user@example.com", "Hello
users!")

 # Ensure constructor was called once
 MockEmailClient.assert_called_once()

 # Ensure send_email on the instance was called correctly
 instance.send_email.assert_called_once_with(
 "user@example.com",
 "System Announcement",
 "Hello users!",
)

if __name__ == "__main__":
 unittest.main()
#@ | Out-File -Encoding UTF8 EXP7\test_notification.py

```

What you now have:

- **Dependency injection** via UserNotifier(email\_client)
- **Mocking techniques:**
  - MagicMock(spec=EmailClient) – mock that behaves like EmailClient
  - side\_effect to simulate an exception
  - @patch("EXP7.notification.EmailClient") to replace the class in the module

### STEP3: Run the tests

```
cd "B:\College\Study Materials\SEM_7\Software Engineering\LAB
\GIT\SE_LABS"
```

```
python -m unittest -v EXP7.test_notification
```

```
>> python -m unittest -v EXP7.test_notification
>>
test_send_system_announcement_uses_email_client (EXP7.test_notification.TestSystemAnnouncement.test_send_system_announcement_uses_email_client) ... ok
test_email_client_raises_error (EXP7.test_notification.TestUserNotifier.test_email_client_raises_error) ... ok
test_send_password_reset_with_link (EXP7.test_notification.TestUserNotifier.test_send_password_reset_with_link) ... ok
test_send_welcome_email_uses_email_client (EXP7.test_notification.TestUserNotifier.test_send_welcome_email_uses_email_client) ... ok

Ran 4 tests in 0.003s

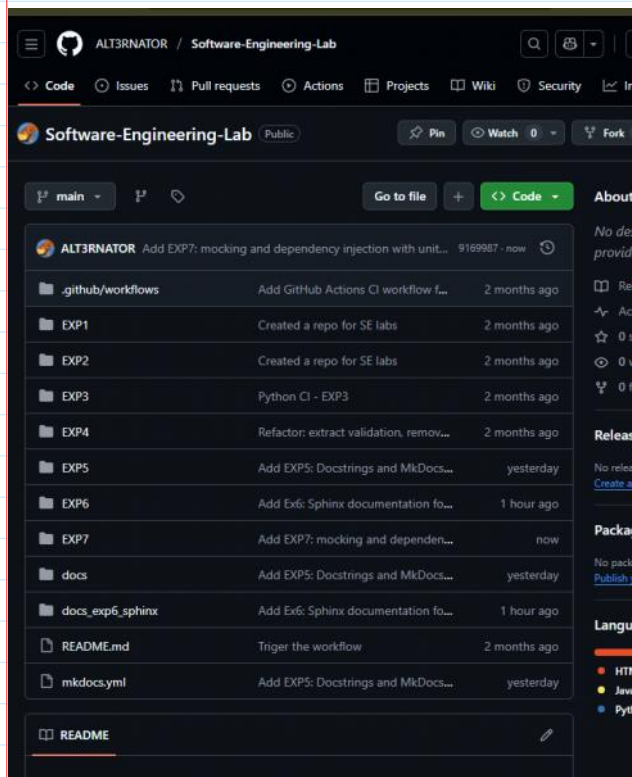
OK
```

### STEP4: Push to Git

```
git add EXP7
```

```
git commit -m "Add EXP7: mocking and dependency injection with
unittest.mock"
```

```
git push origin main
```



## EXP8

24 November 2025 07:27 PM

**AIM:** Implement mocking and dependency injection techniques in python using libraries unittest.mock or dependency-injector.

### Part (b) – Agile Effort Estimation

**Given:**

- Total Story Points = 100 SP
- Team Velocity = 20 SP / Sprint
- Sprint Duration = 2 Weeks
- 1 Month = 4 Weeks

#### i) Number of sprints required

$$\text{Sprints} = 100 / 20 = 5$$

**Number of Sprints = 5**

#### ii) Total effort in weeks

$$5 \text{ Sprints} \times 2 \text{ weeks} = 10 \text{ weeks}$$

**Total Effort = 10 Weeks**

#### iii) Effort in person-months

$$10 \text{ weeks} / 4 = 2.5 \text{ person-months}$$

**Total Effort = 2.5 Person-Months**

### Part (a) – Sprint Planning for 100 Story Points

#### Overall Project Plan

| Sprint   | Duration  | Story Points | Focus                           |
|----------|-----------|--------------|---------------------------------|
| Sprint 1 | Weeks 1–2 | 20 SP        | Project foundation & core setup |
| Sprint 2 | Weeks 3–4 | 20 SP        | Core module development         |

|          |            |       |                                 |
|----------|------------|-------|---------------------------------|
| Sprint 3 | Weeks 5–6  | 20 SP | Feature enhancement & usability |
| Sprint 4 | Weeks 7–8  | 20 SP | Advanced features & security    |
| Sprint 5 | Weeks 9–10 | 20 SP | Optimization & final delivery   |

## Detailed Sprint Breakdown

### Sprint 1 (20 SP) – Weeks 1–2

**Goal:** Establish foundations and implement core architecture.

| Task                                         | Story Points |
|----------------------------------------------|--------------|
| Set up development environment               | 3 SP         |
| Create base project structure / architecture | 5 SP         |
| Implement authentication module              | 8 SP         |
| Basic UI layout and navigation               | 4 SP         |
| <b>Total</b>                                 | <b>20 SP</b> |

### Sprint 2 (20 SP) – Weeks 3–4

**Goal:** Build core functional modules.

| Task                              | Story Points |
|-----------------------------------|--------------|
| Implement User Profile Management | 7 SP         |
| API integration for core data set | 8 SP         |
| Basic error handling & logging    | 3 SP         |
| Unit test suite setup             | 2 SP         |
| <b>Total</b>                      | <b>20 SP</b> |

### Sprint 3 (20 SP) – Weeks 5–6

**Goal:** Enhance features and improve usability.

| Task                                  | Story Points |
|---------------------------------------|--------------|
| Search and filtering functionality    | 8 SP         |
| Advanced validation and form handling | 5 SP         |
| UI/UX improvements & responsiveness   | 4 SP         |
| Extend unit tests for new features    | 3 SP         |
| <b>Total</b>                          | <b>20 SP</b> |

### **Sprint 4 (20 SP) – Weeks 7–8**

**Goal:** Add advanced and security features.

| <b>Task</b>                      | <b>Story Points</b> |
|----------------------------------|---------------------|
| Dashboard with key metrics       | 7 SP                |
| Export functionality (CSV/PDF)   | 5 SP                |
| Role-Based Access Control (RBAC) | 5 SP                |
| Enhanced logging & monitoring    | 3 SP                |
| <b>Total</b>                     | <b>20 SP</b>        |

### **Sprint 5 (20 SP) – Weeks 9–10**

**Goal:** Optimization, testing, and final deployment readiness.

| <b>Task</b>                    | <b>Story Points</b> |
|--------------------------------|---------------------|
| Performance optimization       | 6 SP                |
| Bug fixing and polishing       | 6 SP                |
| End-to-end testing             | 4 SP                |
| User & developer documentation | 4 SP                |
| <b>Total</b>                   | <b>20 SP</b>        |

### **Final Summary**

| <b>Parameter</b>        | <b>Value</b>      |
|-------------------------|-------------------|
| Total Story Points      | 100 SP            |
| Team Velocity           | 20 SP / Sprint    |
| Number of Sprints       | 5                 |
| Sprint Duration         | 2 Weeks           |
| Total Project Duration  | 10 Weeks          |
| Effort in Person-Months | 2.5 Person-Months |